BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH

BÁO CÁO TIỂU LUẬN

ĐỀ TÀI

ỨNG DỤNG MÔ HÌNH PP-OCRv6 TRONG NHẬN DẠNG SỐ ĐIỆN THOẠI NGƯỜI NHẬN CHO HỆ THỐNG SMARTLOCKER

GVHD: ThS. Huỳnh Hoàng Hà

SVTH:	MSSV
Nguyễn Thái Anh	23139002
Vũ Quốc Bảo	23139004
Nguyễn Võ Thái Sơn	23139040
Trần Ngọc Thiện	23139045
Đỗ Hoàng Xuân Tiến	23139047

TP. Hồ Chí Minh, 10/2026

MỤC LỤC

MỤC LỤC ..... **i**

DANH MỤC BẢNG BIỂU ..... **iii**

DANH MỤC HÌNH ẢNH ..... **iv**

DANH MỤC TỪ VIẾT TẮT ..... **v**

GIỚI THIỆU **1**

1. Mục tiêu và nhiệm vụ đề tài **1**

2. Phương pháp thực hiện **1**

3. Bố cục báo cáo **1**

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ BÀI TOÁN **2**

1.1. Bài toán nhận dạng số điện thoại người nhận **2**

1.2. Cơ sở OCR và PP-OCRv6 ..... **3**

CHƯƠNG 2. DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH **6**

2.1. Dữ liệu MC-OCR và thiết kế tập huấn luyện **6**

2.2. Quy trình huấn luyện Recognizer **8**

CHƯƠNG 3. XÂY DỰNG VÀ TÍCH HỢP MÔ HÌNH AI **12**

3.1. Chuyển đổi sang ONNX và quản lý mô hình **12**

3.2. Kiến trúc và luồng xử lý Android **13**

3.3. Thuật toán xác định người nhận V4 ..... **15**

3.4. Xử lý ảnh nghiêng và xoay **18**

CHƯƠNG 4. KIỂM THỬ, TỐI ƯU VÀ TRIỂN KHAI **20**

4.1. Phương pháp kiểm thử mô hình **20**

4.2. Tối ưu hiệu năng trên thiết bị **22**

4.3. Triển khai và bảo mật **23**

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **26**

5.1. Kết quả đạt được **26**

5.2. Hướng phát triển **26**

TÀI LIỆU THAM KHẢO.....27

PHỤ LỤC A. NỘI DUNG NOTEBOOK HUẤN LUYỆN.....28

PHỤ LỤC B. GIẢ MÃ LUỒNG NHẬN DIỆN.....29

DANH MỤC BẢNG BIỂU

Bảng 1.1. Tiêu chí đánh giá mô-đun AI 3
Bảng 2.1. Vai trò của các nhóm dữ liệu 8
Bảng 2.2. Thông số huấn luyện Recognizer 11
Bảng 3.1. Định danh mô hình triển khai 13
Bảng 3.2. Vai trò các file tích hợp AI 15
Bảng 3.3. Trạng thái kết quả của V4 18
Bảng 4.1. Bộ kiểm thử độ bền vững 20
Bảng 4.2. Các phương án tối ưu đã khảo sát 23
Bảng A.1. Các công đoạn notebook huấn luyện 28

DANH MỤC HÌNH ẢNH

Hình 2.1. Quy trình xây dựng mô hình PP-OCRv6 8
Hình 3.1. Luồng xử lý OCR trên thiết bị Android 14

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Giải thích
AI	Artificial Intelligence — trí tuệ nhân tạo
OCR	Optical Character Recognition — nhận dạng ký tự quang học
ONNX	Open Neural Network Exchange — định dạng biểu diễn mô hình
ORT	ONNX Runtime — bộ thực thi mô hình ONNX
CTC	Connectionist Temporal Classification — phương pháp học chuỗi không cần căn chỉnh nhãn theo từng bước thời gian
ROI	Region of Interest — vùng quan tâm
MC-OCR	Bộ dữ liệu hóa đơn tiếng Việt dùng trong notebook
FP32	Float 32-bit
SĐT	Số điện thoại
V4	Phiên bản thuật toán lựa chọn phone theo điểm và ngữ nghĩa
FCM	Firebase Cloud Messaging
UI	User Interface — giao diện người dùng

GIỚI THIỆU

1. Mục tiêu và nhiệm vụ đề tài

Hệ thống SmartLocker hướng tới việc hỗ trợ quá trình gửi và nhận hàng bằng tủ thông minh. Trong quy trình gửi hàng, thông tin quan trọng cần xác định từ nhãn vận chuyển là số điện thoại người nhận. Việc nhập tay kéo dài thời gian thao tác và có nguy cơ nhầm số của người gửi, người nhận hoặc tổng đài vận chuyển.

Mục tiêu của chuyên đề là xây dựng mô-đun OCR hai giai đoạn có khả năng đọc các dòng chữ trên nhãn giao hàng, xác định số điện thoại người nhận bằng quy tắc ngữ nghĩa và tích hợp vào ứng dụng React Native chạy trên thiết bị Android.

2. Phương pháp thực hiện

Nhóm sử dụng Detector PP-OCRv6 Small pretrained để xác định vùng văn bản; Recognizer PP-OCRv6 Small được fine-tune trên dữ liệu MC-OCR 2021 và dữ liệu tăng cường số điện thoại. Các mô hình suy luận được chuyển sang ONNX và thực thi bằng ONNX Runtime trong mô-đun Kotlin Native.

Sau bước nhận dạng chữ, thuật toán Receiver Selector V4 phân tích các số điện thoại ứng viên theo mẫu số Việt Nam, nhãn ngữ nghĩa và vị trí dòng. Chương trình bổ sung tự hiệu chỉnh độ nghiêng và xử lý các hướng xoay trong trường hợp kết quả chưa đủ tin cậy.

3. Bố cục báo cáo

Báo cáo gồm năm chương. Chương 1 trình bày bài toán và cơ sở OCR; Chương 2 phân tích dữ liệu và huấn luyện; Chương 3 mô tả mô hình ONNX và quá trình tích hợp Android; Chương 4 trình bày phương pháp kiểm thử và lựa chọn tối ưu; Chương 5 tổng kết kết quả và định hướng phát triển.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ BÀI TOÁN

1.1. Bài toán nhận dạng số điện thoại người nhận

1.1.1. Bối cảnh ứng dụng

Trong mô hình giao nhận thông thường, nhân viên giao hàng hoặc người vận hành nhập thủ công số điện thoại của người nhận để ánh xạ kiện hàng vào tài khoản, tủ hoặc thông báo. Việc nhập tay không chỉ kéo dài thao tác mà còn dễ nhầm số khi thông tin trên nhãn nhiều, bố cục phức tạp hoặc ảnh không rõ. SmartLocker hướng tới một chu trình giao nhận trong đó phần AI xử lý thông tin trên bill, phần mềm ứng dụng xác nhận kết quả và backend tiếp tục điều phối tủ.

Một bill có thể chứa nhiều chuỗi gồm 10 chữ số, nhiều nhãn viết khác nhau như "Đến", "Người nhận", "Receiver", "SDT", "SĐT", "Điện thoại", trong khi một phần số bị OCR nhầm ký tự hoặc cắt thành nhiều box. Vì vậy, lựa chọn số điện thoại đúng phải dựa trên nhiều nguồn tín hiệu: mẫu chuỗi số, ngữ nghĩa dòng chữ và vị trí tương đối giữa các box. Dùng regex để lấy chuỗi số đầu tiên là cách đơn giản nhưng không an toàn đối với bill có cả số người gửi và người nhận.

SmartLocker không hướng đến việc số hóa toàn bộ dữ liệu thương mại trên nhãn. Đầu ra cần thiết nhất đối với phần AI là số điện thoại người nhận, mức tin cậy hoặc trạng thái chưa chắc chắn, và các thông tin phụ phục vụ kiểm tra. Bài toán có thể được hiểu là một tác vụ OCR kết hợp suy luận quy tắc có điều kiện, chứ không phải phân loại ảnh đơn thuần.

1.1.2. Phát biểu bài toán

Đầu vào là một ảnh RGB hoặc JPEG chụp nhãn giao hàng từ camera điện thoại. Ảnh có thể bị xoay 0/90/180/270 độ, nghiêng trong mặt phẳng, lệch phối cảnh, chứa vùng nền ngoài bill hoặc có chữ nhỏ. Đầu ra là một SĐT người nhận hợp lệ theo quy tắc áp dụng trong hệ thống; khi tín hiệu yếu, hệ thống cần báo chưa chắc chắn thay vì đưa ra số người gửi do nhầm lẫn.

Biểu diễn hệ thống theo các phép biến đổi: $I \rightarrow P(I) \rightarrow D(P(I)) \rightarrow R(\text{Crop}(\text{box})) \rightarrow V4(\text{lines}) \rightarrow p^*$. Trong đó P là tiền xử lý ảnh; D là Detector trả danh sách box; R là Recognizer trả text và confidence; V4 là thuật toán trích xuất và chấm điểm; p* là số điện thoại người nhận được lựa chọn. Mỗi khâu được đánh giá riêng để dễ xác định nguyên nhân của lỗi đầu-cuối.

1.1.3. Mục tiêu kỹ thuật và tiêu chí thành công

Bảng 1.1. Tiêu chí đánh giá mô-đun AI

Nhóm tiêu chí	Yêu cầu đặt ra	Cách đo
Độ đúng	Chọn SĐT người nhận, không nhầm người gửi	Exact match trên tập nhãn có ground truth
Độ bao phủ	Có thể quyết định đúng hoặc báo chưa chắc chắn	Coverage và tỷ lệ UNCERTAIN
Tốc độ	Trả kết quả nhanh trên điện thoại Android	Thời gian từ lúc chụp đến lúc hiện kết quả
Robustness	Chịu được quay ngang, nghiêng, ảnh khó	Tỷ lệ đúng theo từng dạng ảnh
Tính tái lập	Có notebook, code, cấu hình, mô tả dataset	Kiểm tra đầu vào, tham số, phiên bản, đầu ra
Khả năng tích hợp	Kết quả từ AI đi vào form gửi hàng	Demo luồng ShipperScreen và nghiệp vụ

1.1.4. Giới hạn của mô-đun AI

Phạm vi tài liệu tập trung vào nhận dạng bill, xuất model và lựa chọn SĐT. Các thao tác quản lý tủ, mở khóa, điều khiển ESP32, đăng nhập, OTP và gửi thông báo thuộc các mô-đun phần mềm/phần cứng khác. Tuy nhiên, phần AI phải tạo được giao diện tích hợp phù hợp: ảnh vào, kết quả phone ra, ngoại lệ rõ ràng khi không đọc được, và khả năng sửa thủ công để tránh sai đơn hàng.

Một điểm cần phân biệt: kiểm thử crop nhận dạng SĐT, kiểm thử OCR trên cả bill và kiểm thử thời gian trên Android là ba cấp độ khác nhau. Thành công ở cấp độ đầu không đủ chứng minh thành công ở cấp độ cuối. Vì vậy báo cáo tách phần training/evaluation offline khỏi ứng dụng thực tế.

1.2. Cơ sở OCR và PP-OCRv6

1.2.1. Khái niệm OCR và vì sao dùng hai mô hình

OCR biến các ký tự trong ảnh thành văn bản có thể xử lý tự động. Với ảnh bill chụp tự do, vị trí chữ không biết trước; nếu đưa toàn bộ ảnh cho một bộ đọc ký tự dạng text-line, mô hình sẽ không biết phải đọc dòng nào. Kiến trúc hai bước tách phát hiện vị trí chữ (text detection) và nhận dạng nội dung (text recognition) giúp giải quyết vấn đề đó.

Detector nhận một ảnh đã chuẩn hóa và ước lượng vùng ảnh chứa văn bản. Kết quả là tập các hình chữ nhật hoặc polygon, có tọa độ và điểm phát hiện. Mỗi box được cắt ảnh (crop), có thể hiệu chỉnh hướng, rồi đưa vào Recognizer. Recognizer tạo chuỗi ký tự tương ứng, thông thường kèm thông tin confidence. Cách tách hai bước cho phép tối ưu detector cho bố cục và recognizer cho ngôn ngữ/chuỗi ký tự.

1.2.2. PP-OCRv6 Small và quyết định sử dụng pretrained

Notebook sử dụng họ PP-OCRv6 Small trong hệ sinh thái PaddleOCR. Đây là điểm xuất phát pretrained, không phải mạng nơ-ron được viết lại từ đầu. Sử dụng pretrained hợp lý với đồ án vì nó tận dụng tri thức hình ảnh/chữ đã học, giảm nhu cầu dữ liệu lớn và số epoch huấn luyện. Tuy nhiên, khả năng đọc ký tự trên hóa đơn tiếng Việt và SĐT vẫn cần được kiểm tra trên dữ liệu đúng miền ứng dụng. [1]

Trong quy trình thực hiện, Detector PP-OCRv6 Small sử dụng trọng số pretrained và được xuất phục vụ suy luận; chỉ Recognizer được fine-tune trên dữ liệu hóa đơn và dữ liệu số điện thoại. Cách tổ chức này giúp tập trung tài nguyên huấn luyện vào khả năng nhận dạng chuỗi ký tự, đồng thời duy trì bộ phát hiện chữ ban đầu.

1.2.3. Detector: bản đồ xác suất, ngưỡng và hộp chữ

Trong triển khai Android stable, bộ hậu xử lý Detector sử dụng ngưỡng pixel 0,20 và ngưỡng box 0,45. Hai đại lượng có vai trò khác nhau: ngưỡng pixel hỗ trợ tạo mask vùng có khả năng chứa chữ; ngưỡng box loại các vùng có điểm tổng hợp thấp. Nếu ngưỡng quá cao, chữ nhỏ có thể bị bỏ sót; nếu ngưỡng quá thấp, số box rác tăng, khiến Recognizer phải xử lý thêm.

Kích thước ảnh làm việc được giới hạn trong khoảng tối đa 1.024 × 1.440 px để kiểm soát lượng phép toán khi chạy native. Đây là mức trần dùng trong mã nguồn, không có nghĩa mọi ảnh bị ép về đúng một độ phân giải. Tỷ lệ ảnh và cách resize ảnh hưởng đến độ cao chữ còn lại; chữ quá nhỏ có thể làm OCR thiếu ký tự. Vì vậy việc giảm size phải được cân bằng với chất lượng vùng SĐT.

1.2.4. Recognizer và cơ chế giải mã CTC

Recognizer nhận ảnh crop dòng chữ, chuẩn hóa thành tensor 3 kênh với chiều cao đặc trưng 48 pixel và chiều rộng xử lý chuẩn 320 pixel ở bản stable. Mô hình tạo dự đoán theo từng bước thời gian trên trục ngang ảnh, mỗi bước là phân bố lớp ký tự

từ vocabulary. Với chuỗi số điện thoại, một ký tự có thể xuất hiện qua nhiều timestep liên tiếp nên cần bước giải mã chuỗi CTC.

CTC sử dụng lớp blank để mô hình biểu diễn sự chuyển tiếp giữa các ký tự. Khi giải mã greedy, ta chọn lớp có xác suất cao nhất tại mỗi timestep, bỏ các lặp liên tiếp rồi bỏ blank. Ví dụ chuỗi lớp [blank, 0, 0, blank, 3, 3, 2] sau gộp lặp và loại blank thành "032". Nếu bỏ blank không đúng thứ tự, các ký tự trùng liên tiếp trong SĐT có thể bị xóa nhầm. [8]

Bản TopK-F32 được sử dụng để giảm chi phí trao đổi/giải mã tensor đầu ra. Cần phân biệt tối ưu này với việc thu nhỏ bộ ký tự: TopK-F32 không có nghĩa Recognizer đã được train với ít lớp hơn; nó chủ yếu thay cách cung cấp các giá trị tốt nhất trong tensor inference. Phiên bản thử nghiệm prune vocabulary và thay width input đã được loại khỏi bản stable vì không có chứng cứ đạt đồng thời tốc độ và độ đúng.

1.2.5. ONNX và suy luận on-device

ONNX là biểu diễn trung gian giúp tách quá trình huấn luyện bằng PaddleOCR khỏi runtime inference Android. Sau khi export ở dạng Paddle inference legacy IR, notebook tạo các model ONNX. Thiết bị di động không cần chạy lại môi trường huấn luyện và không tải dataset; nó chỉ cần weights ONNX tương thích, dictionary và runtime hỗ trợ operator của graph. [9]

Suy luận on-device có hai lợi thế đáng kể: dữ liệu ảnh bill có thể được xử lý trên điện thoại và thời gian không phụ thuộc vào việc tải ảnh lên máy chủ. Tuy vậy, thời gian xử lý vẫn phụ thuộc chipset/CPU, kích thước ảnh, số box, số batch và việc phải fallback nhiều góc. Mọi kết quả tốc độ cần ghi rõ thiết bị, build release/debug, điều kiện ánh sáng và phạm vi đo.

CHƯƠNG 2. DỮ LIỆU VÀ HUẤN LUYỆN MÔ HÌNH

2.1. Dữ liệu MC-OCR và thiết kế tập huấn luyện

2.1.1. Nguồn dữ liệu MC-OCR

Trong notebook chính, tập dữ liệu được gọi là "Vietnamese Receipts MC_OCR 2021". Cell 3 tìm file mcocr_train_df.csv trong /kaggle/input, sau đó kiểm tra những cột bắt buộc gồm img_id, anno_polygons và anno_texts. Ảnh được đọc từ thư mục train_images của dataset đã gắn vào Kaggle. Đây là nguồn dữ liệu thật dùng để tạo mẫu huấn luyện Recognizer, không phải bộ robustness 36 ảnh và không phải bộ unseen 6 ảnh. [7]

Bộ dữ liệu sử dụng trong notebook được xác định thông qua hai thành phần mcocr_train_df.csv và train_images trong thư mục đầu vào Kaggle. Khi triển khai lại thực nghiệm, cần lựa chọn đúng phiên bản dữ liệu để bảo đảm cấu trúc annotation tương ứng với mã xử lý.

2.1.2. Từ annotation polygon sang text-line crop

Nhóm không dùng cả ảnh hóa đơn làm nhãn duy nhất cho Recognizer. Mỗi ảnh được kết hợp với polygon và chuỗi văn bản tương ứng; Cell 5 chuyển từng vùng polygon thành crop dòng chữ thông qua cách sắp thứ tự bốn góc và phép biến đổi phối cảnh. Nếu annotation chỉ có bounding box, mã hỗ trợ tạo tứ giác thay thế. Crop quá nhỏ hoặc không hợp lệ được loại khỏi quá trình tạo dữ liệu.

Phép biến đổi phối cảnh được biểu diễn theo tọa độ đồng nhất:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Trong đó H là ma trận 3 × 3, λ là hệ số tỷ lệ khác 0; ma trận được xác định từ bốn cặp điểm nguồn và đích. Sau warpPerspective, vùng chữ được đưa về ảnh chữ dễ đọc hơn. Cell 5 còn kiểm tra crop đứng bất thường để quay lại hướng ngang.

Chuỗi nhãn được chuẩn hóa Unicode NFC, loại ký tự điều khiển và giữ lại những ký tự nằm trong dictionary Recognizer. Giới hạn độ dài nhãn MAX_TEXT_LEN=25 được đặt theo cấu hình Recognizer. Mỗi mẫu huấn luyện được ghi theo định dạng <đường_dẫn_crop><TAB><chuỗi_nhãn>, giúp PaddleOCR loader biết ảnh nào ứng với text nào.

2.1.3. Chia train/validation/test theo hóa đơn gốc

Trong dữ liệu OCR, nhiều crop khác nhau có thể xuất phát từ cùng một hóa đơn. Nếu chia crop ngẫu nhiên, các dòng từ một ảnh có thể đồng thời nằm ở train và test, làm ước lượng khả năng tổng quát hóa bị lạc quan. Notebook giải quyết bằng cách xáo danh sách hóa đơn theo seed và chia cấp document: 80% train, 10% validation, 10% test; sau đó mới gán toàn bộ crop của cùng hóa đơn vào đúng split.

Thao tác này giảm nguy cơ rò rỉ do cùng bố cục, font, điều kiện sáng của một bill xuất hiện trong cả train/test. Tuy nhiên, nó không bảo đảm hoàn toàn độc lập giữa những bill khác nhau nếu nhiều ảnh được sinh từ cùng mẫu. Vì vậy ngoài test MC-OCR, nhóm cần có bộ kiểm thử bill vận chuyển riêng và tập unseen để đánh giá tính tổng quát.

2.1.4. Dữ liệu tăng cường số điện thoại

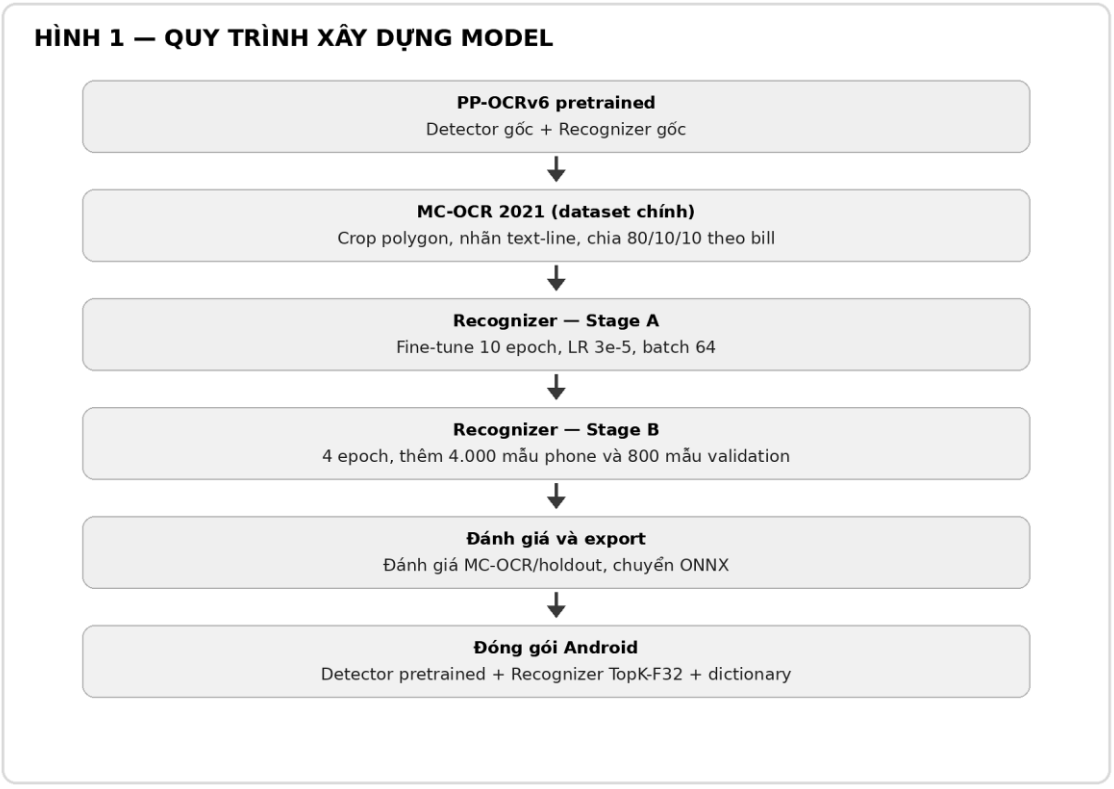
Hóa đơn thông thường có nhiều loại số, song bài toán SmartLocker chú trọng chuỗi số điện thoại Việt Nam. Do đó notebook bật USE_PHONE_AUG=True và tạo thêm 4.000 ảnh dòng số để train, 800 ảnh validation cho Stage B. Mẫu tổng hợp có thể được biến đổi về font, kích thước, độ mờ, nhiễu, phối cảnh và suy giảm độ phân giải theo các đoạn mã trong Cell 10.

Mục đích augmentation là làm Recognizer quen hơn với cấu trúc số dài và những cách trình bày điển hình của dòng điện thoại, chứ không dùng synthetic phone để thay thế dữ liệu thực. Vì vậy đánh giá cuối có hai phần: MC-OCR test thật và synthetic phone holdout. Thành công trên holdout tổng hợp chỉ cho thấy khả năng đọc kiểu chuỗi tương tự; chưa đủ suy ra accuracy trên bill giao hàng chụp bằng điện thoại.

2.1.5. Các nhóm dữ liệu sử dụng

Bảng 2.1. Vai trò của các nhóm dữ liệu

Nhóm dữ liệu	Vai trò trong hệ thống	Quy mô hoặc đặc điểm
MC-OCR 2021	Fine-tune Recognizer; chia dữ liệu theo hóa đơn gốc	Tập hóa đơn tiếng Việt có polygon và nhãn chữ
Synthetic phone	Tăng cường Stage B và validation	4.000 mẫu train; 800 mẫu validation theo cấu hình
Robustness	Đánh giá ảnh gốc và biến thể xoay/nghiêng	36 trường hợp từ 4 ảnh nguồn
Unseen	Khảo sát nhãn khác bố cục với dữ liệu không dùng train	6 ảnh nhãn kiểm thử



Hình 2.1. Quy trình xây dựng mô hình PP-OCRv6

2.2. Quy trình huấn luyện Recognizer

2.2.1. Nguyên tắc xây dựng pipeline có thể tái lập

Notebook huấn luyện được tổ chức 14 cell để tách các nhiệm vụ cài môi trường, kiểm tra dữ liệu, tạo crop, huấn luyện, đánh giá và export. Việc tách cell làm rõ đầu ra trung gian, giảm việc huấn luyện lại không cần thiết nếu một bước export bị lỗi. Việc

chạy lại thực nghiệm cần dữ liệu MC-OCR tương ứng, pretrained weights và môi trường Kaggle có các thư viện phụ thuộc phù hợp.

Mã bắt đầu với seed 20261003 để cố định việc xáo dữ liệu và nhiều phép sinh ngẫu nhiên. Tuy nhiên, seed không đủ bảo đảm kết quả huấn luyện bit-for-bit giữa các GPU, phiên bản CUDA, thư viện augmentation hay loader song song. Để tái lập học thuật, cần lưu thêm version của PaddleOCR, PaddlePaddle, pretrained checkpoints, môi trường và log huấn luyện.

2.2.2. Thiết lập môi trường, tải pretrained và chuẩn hóa cấu hình

Cell 1 xác định tham số chung: USE_PHONE_AUG=True, REC_REAL_EPOCHS=10, REC_PHONE_EPOCHS=4, PHONE_AUG_TRAIN=4000, PHONE_AUG_VAL=800. Cell 2 thiết lập PaddleOCR v3.7.0 bằng cách tải release archive từ GitHub thay cho clone trực tiếp một tag, kết hợp cài thư viện Paddle GPU thích hợp. Cell 4 tải trọng số PP-OCRv6 Small Detector và Recognizer chính thức. [2]

Quy trình áp dụng transfer learning thay vì huấn luyện từ đầu: sử dụng tri thức từ pretrained model, cập nhật trọng số Recognizer trên miền hóa đơn. Nền tảng này tiết kiệm thời gian đáng kể, giảm yêu cầu dữ liệu gốc và tạo cơ sở tốt hơn để tiến tới ứng dụng trên thiết bị thực.

2.2.3. Huấn luyện Stage A trên dữ liệu MC-OCR thật

Cell 8 xây dựng file YAML từ cấu hình configs/rec/PP-OCRv6/PP-OCRv6_small_rec.yml. Các trường chính được sửa gồm: use_gpu=True, distributed=False, epoch_num=10, pretrained_model trỏ tới recognizer pretrained, checkpoints=None, learning_rate=3×10^{-5}, warmup_epoch=1, batch_size_per_card=64. File train.txt và val.txt được trỏ tới những crop đã tạo ở Cell 5.

Đặt checkpoints=None là điểm cần giải thích: thao tác này bắt đầu một phiên fine-tune mới trên pretrained weights mà không tiếp tục toàn bộ trạng thái optimizer/epoch từ một phiên trước. Tập validation dùng để xem chất lượng qua các kỳ đánh giá, trong khi tập test không được dùng để chọn cấu hình ngay trong quá trình train. Khi Stage A kết thúc, Cell 9 tìm checkpoint tốt nhất và thực hiện eval trên MC-OCR test.

2.2.4. Huấn luyện Stage B có tăng cường phone

Cell 10 tạo ảnh phone bổ sung, cập nhật danh sách nhãn và cấu hình Stage B. Stage này bắt đầu từ best weights của Stage A nhưng không resume optimizer/epoch cũ; số epoch mới là 4. Logic này tránh trường hợp epoch đã lưu của Stage A lớn hơn số epoch Stage B, khiến hệ thống không còn epoch thực sự để huấn luyện.

Stage B là điều chỉnh có mục tiêu (targeted fine-tuning), không đồng nghĩa học được quy tắc chọn số người nhận. Recognizer chỉ xử lý chuỗi văn bản ở cấp crop. Việc hiểu "người nhận" hay "người gửi" vẫn là nhiệm vụ của thuật toán V4 ở tầng hậu xử lý. Sự phân biệt này làm rõ hai vai trò: mô hình AI học cách đọc chữ từ dữ liệu; tầng chọn số điện thoại đúng đối tượng kết hợp ngữ nghĩa và luật.

2.2.5. Đánh giá và export checkpoint

Cell 11 tạo cấu hình đánh giá cuối đối với MC-OCR test thật, và nếu bật augmentation thì thêm cấu hình đánh giá trên tập synthetic phone holdout. Cell 12 xuất Recognizer sang Paddle inference legacy IR. Cell 7 trước đó đã xuất Detector pretrained theo dạng tương tự. Cell 13 dùng Paddle2ONNX để chuyển Detector/Recognizer sang ONNX; để tương thích môi trường, notebook có bước tạo converter Python 3.12 độc lập. [2]

Cell 14 tạo gói mô hình suy luận gồm Detector, Recognizer, từ điển ký tự và các tệp cấu hình có liên quan. Notebook đồng thời tổ chức đầu ra phục vụ kiểm tra checkpoint và tái sử dụng trong môi trường Android.

2.2.6. Tham số huấn luyện

Bảng 2.2. Thông số huấn luyện Recognizer

Thông số	Giá trị trong notebook	Ý nghĩa
Seed	20261003	Cố định xáo split và sinh mẫu
Recognizer gốc	PP-OCRv6_small_rec pretrained	Khởi tạo transfer learning
Detector	PP-OCRv6_small_det pretrained	Giữ trọng số; chỉ export
Stage A	10 epoch	Fine-tune MC-OCR
Stage B	4 epoch	Fine-tune bổ sung phone
LR Stage A	3×10^{-5}	Cập nhật trọng số bảo thủ
Batch train Stage A	64 / thiết bị	Batch loader
Synthetic train	4.000 mẫu	Tăng cường chuỗi phone
Synthetic validation	800 mẫu	Holdout tăng cường
Split MC-OCR	80 / 10 / 10 theo bill	Hạn chế leakage
Max text length	25	Giới hạn nhãn nguồn

CHƯƠNG 3. XÂY DỰNG VÀ TÍCH HỢP MÔ HÌNH AI

3.1. Chuyển đổi sang ONNX và quản lý mô hình

3.1.1. Phân biệt checkpoint huấn luyện và mô hình suy luận

Checkpoint Paddle (.pdparams) lưu trọng số phục vụ huấn luyện/đánh giá trong hệ sinh thái Paddle; model ONNX là dạng suy luận triển khai cho runtime tương thích. Khi mô hình được đưa vào Android assets, người dùng không cần cài Kaggle, PaddleOCR hay dataset. Tuy nhiên muốn huấn luyện lại, cần mã nguồn notebook, dataset và pretrained/checkpoint tùy giai đoạn.

Phiên bản sử dụng trong ứng dụng là Detector V6 và Recognizer V6 TopK-F32. Tên TopK-F32 thể hiện dạng đầu ra hỗ trợ giải mã trong ứng dụng, không phải cơ chế lượng tử hóa INT8. Bộ từ điển ký tự phải tương thích với đầu ra Recognizer để bảo đảm ánh xạ chỉ số lớp về ký tự chính xác.

3.1.2. Định danh và kiểm tra tính nhất quán mô hình

Một bộ OCR chỉ hoạt động đúng khi model Recognizer và dictionary cùng phiên bản. Nếu thay Recognizer khác mà giữ từ điển không khớp, chỉ số lớp CTC có thể ánh xạ thành ký tự khác, gây sai SĐT hàng loạt dù suy luận không crash. Tương tự, đổi graph TopK, kiểu số hoặc output name có thể làm code Kotlin giải mã sai tensor. Vì vậy quy trình đổi model phải đi theo nhóm model2_recognizer.onnx + ppocr_keys.txt và kiểm tra output contract.

Bảng 3.1 ghi dấu nhận dạng SHA-256 ở dạng rút gọn để phân biệt các thành phần. Khi xác minh tính toàn vẹn, cần đối chiếu đủ 64 ký tự thập lục phân với manifest của đúng phiên bản. Trong quá trình phát triển từng có Static INT8, Dynamic INT8, Progressive ROI và thay width input, nhưng không đưa các candidate này vào nhánh stable vì không có đồng thời bằng chứng về chất lượng và tốc độ đáp ứng yêu cầu.

Bảng 3.1. Định danh mô hình triển khai

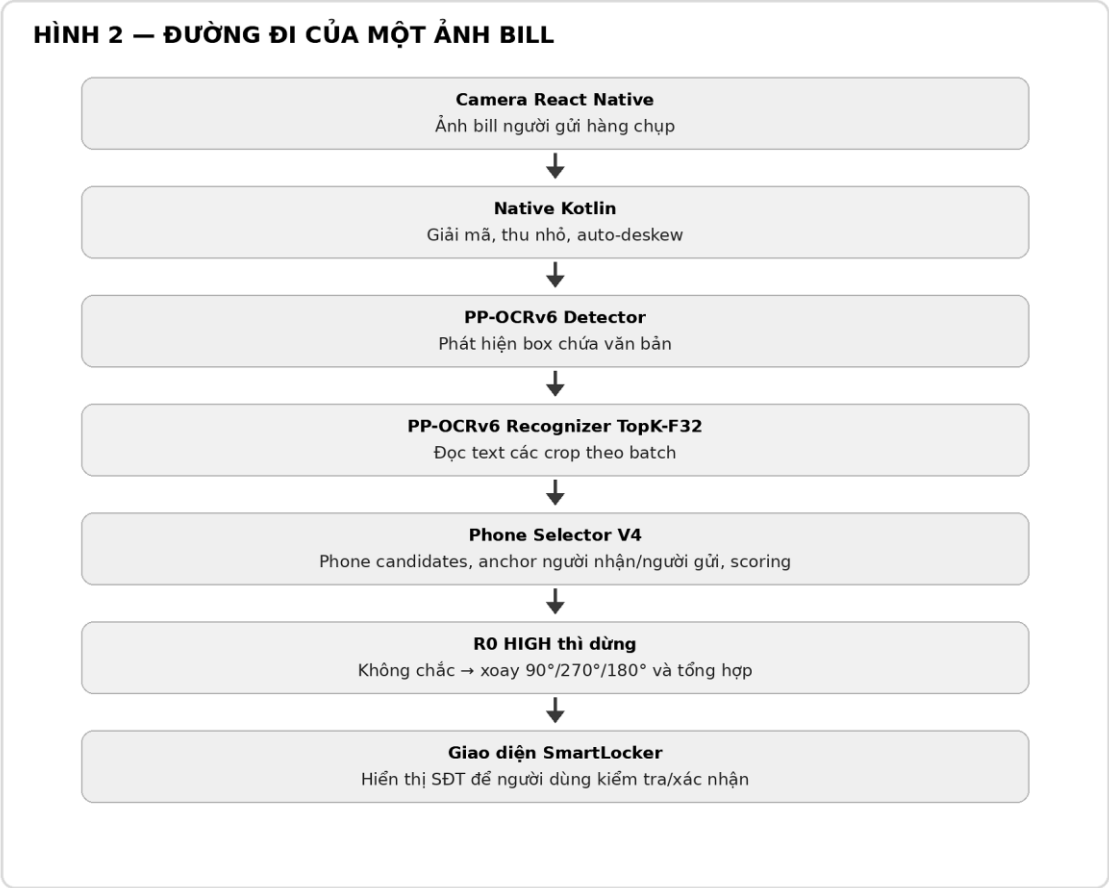
Artifact ổn định	Dung lượng gần đúng	SHA-256 (rút gọn)
Detector V6 ONNX	9,43 MiB	3914f972…ee26a93869940
Recognizer TopK-F32	20,17 MiB	550aae3077…c43da638
ppocr_keys.txt	~73 KB	b5f2bfe2…77e401c5d

3.2. Kiến trúc và luồng xử lý Android

3.2.1. Vai trò của React Native và Kotlin Native

Ứng dụng SmartLocker sử dụng React Native cho màn hình Shipper và nghiệp vụ người dùng. Code src/screens/ShipperScreen.tsx điều khiển camera, quản lý trạng thái đang chụp/đang xử lý và nhận phone để đưa vào form. Code src/ai/smartlockerOcr.ts nhận các dòng OCR, chuẩn hóa và chọn số người nhận theo V4. Phần tiền xử lý và inference ONNX chuyên sâu được đưa xuống Android Native ở SmartLockerOcrNativeModule.kt; SmartLockerOcrNativePackage.kt và MainApplication.kt giúp mô-đun được đăng ký để JavaScript gọi được. [10]

Kiến trúc này vẫn đáp ứng yêu cầu sử dụng React Native: React Native quản lý trải nghiệm và luồng nghiệp vụ, Kotlin làm module tăng tốc tích hợp. Trong kiến trúc này, Kotlin thực hiện các tác vụ tính toán nặng nằm trong luồng ứng dụng React Native, còn giao diện và nghiệp vụ vẫn do React Native quản lý.



Hình 3.1. Luồng xử lý OCR trên thiết bị Android

3.2.2. Tiền xử lý ảnh bằng Native Android

Sau thao tác camera, module native nhận đường dẫn ảnh, giải mã thành Bitmap và giới hạn kích thước ảnh làm việc để tránh cấp phát quá nhiều bộ nhớ. Auto-deskew được thực hiện nhằm sửa lệch góc trong mặt phẳng ảnh trước khi Detector chạy. Ở mã ổn định, vùng góc tìm kiếm là ±35°, bước quét thô 5°, hiệu chỉnh tinh trong bán kính 4°, chỉ áp dụng khi góc đủ đáng kể và lợi ích chấm điểm ảnh vượt ngưỡng. [4]

Điều cần lưu ý: auto-deskew xử lý xoay trong mặt phẳng (roll) và chưa giải quyết hoàn toàn hiện tượng biến dạng phối cảnh khi điện thoại chụp xiên mạnh. Trường hợp phối cảnh nặng, vùng chữ có thể bị co hoặc méo, dẫn đến Detector sai box hoặc Recognizer đọc thiếu. Đây là giới hạn của bước deskew trong pipeline hiện tại.

3.2.3. Phát hiện, crop và batch recognition

Detector tạo danh sách box văn bản từ ảnh đã xử lý. Mỗi box được crop, biến đổi về dạng tensor RGB/BGR theo quy ước model, chuẩn hóa và đưa vào Recognizer. Batch recognition hiện sử dụng REC_BATCH_SIZE=16, nên số lượng box ảnh hưởng

đến số lượt chạy ONNX. Một bill nhiều thông tin có thể tạo số box lớn hơn; tốc độ không tỷ lệ đơn giản với số chữ vì mỗi crop vẫn phải xử lý ở kích thước input chuẩn.

Tối ưu quan trọng của phiên bản native là tránh chuyển lượng lớn tensor từ Kotlin sang JavaScript để giải mã CTC. Module trả về các dòng có text, confidence, vị trí box và profile thời gian. JavaScript tiếp tục dùng vị trí không gian để nhận biết nhãn "người nhận" tương ứng với số nào. Đây là nơi các thông tin hình học từ Detector không bị bỏ phí sau OCR.

3.2.4. Các thành phần phần mềm

Bảng 3.2. Vai trò các file tích hợp AI

File	Vai trò đối với AI	Trách nhiệm khi tích hợp
SmartLockerOcrNativeModule.kt	Decode, deskew, detector, recognizer, profiling	Cần assets đúng version và Android runtime
SmartLockerOcrNativePackage.kt	Đăng ký module native	Giúp bridge nhìn thấy module
MainApplication.kt	Khai báo package	Tích hợp vào ứng dụng
smartlockerOcr.ts	Logic OCR/V4/fallback	Xác định phone đúng người nhận
ShipperScreen.tsx	Camera, kết quả, sửa tay	Gắn chức năng vào UI/đơn gửi
android/app/build.gradle	Phụ thuộc Android	Phiên bản runtime tương thích
model1/model2/ppocr_keys	Model weights và mapping	Phân phối cùng bộ assets hoặc gói mô hình riêng

3.3. Thuật toán xác định người nhận V4

3.3.1. Vì sao cần lớp quyết định sau OCR

Sau khi OCR, danh sách các dòng có thể chứa số điện thoại người gửi, người nhận, hotline, mã vận đơn, mã đơn, ngày giờ, COD và địa chỉ. Nếu chọn chuỗi có độ tin cậy OCR cao nhất, hệ thống dễ nhầm số người gửi với số người nhận. Nếu chọn

chuỗi 10 chữ số đầu tiên, hệ thống sẽ nhạy với thứ tự đọc của Detector. Receiver Selector V4 dùng kết hợp phone validation, từ khóa ngữ nghĩa và quan hệ tọa độ giữa các dòng.

V4 không phải mạng nơ-ron được huấn luyện lại. Nó là thuật toán hậu xử lý có các quy tắc ưu tiên và mức điểm. Do đó, kết quả "HIGH/MEDIUM/LOW" là nhãn tin cậy nội bộ của bộ chọn chứ không phải xác suất đã được hiệu chuẩn trên tập thực nghiệm. Các nhãn này vì thế được diễn giải như mức tin cậy theo luật.

3.3.2. Chuẩn hóa văn bản và tách ứng viên

Tại bước đầu, text được chuẩn hóa để giảm tác động của chữ hoa/thường, khoảng trắng, dấu tiếng Việt và biến thể nhãn. Các chuỗi nghi là phone được trích xuất theo cấu trúc SĐT Việt Nam với những kiểm tra độ dài và tiền tố. Mục đích là giảm số ứng viên rác trước khi thực hiện phân tích ngữ cảnh. Trong code V4, việc tách ứng viên không nên xem như chứng minh duy nhất của tính đúng: vẫn phải kết hợp các nhãn quanh số.

Với nhãn "SĐT", "Phone", "Điện thoại", "Người nhận", "Đến", hệ thống đánh giá cả độ gần về vị trí và mức confidence OCR. Những trường hợp một ký tự bị nhầm do OCR có thể cần quy tắc sửa đổi có kiểm soát, nhưng không được tự ý thay nhiều chữ số vì sẽ làm tăng nguy cơ tạo ra SĐT hợp lệ giả.

3.3.3. Semantic anchors: người nhận đối lập người gửi

Một semantic anchor là dòng OCR có tín hiệu ngữ nghĩa, chẳng hạn "NGƯỜI NHẬN" hoặc "NGƯỜI GỬI". Dựa trên tọa độ box, hệ thống suy ra vùng nào có khả năng thuộc block người nhận và vùng nào thuộc block người gửi. Điểm của một phone được tăng nếu gần anchor người nhận và giảm nếu gần anchor người gửi; các cấu trúc bill nhiều cột cần chú ý thêm tính tương thích theo trục x để tránh ghép nhầm số ở cột trái vào nhãn bên phải.

Việc phân biệt hai tập anchor giải quyết tình huống khó nhất trong đề tài: cả hai số đều hợp lệ, có cùng số chữ số và có thể cùng confidence OCR cao. Nhiệm vụ V4 chính là chọn đúng vai trò nghiệp vụ chứ không chỉ nhận diện một dạng chuỗi ký tự.

3.3.4. Hàm điểm ứng viên và margin

Diễn giải tổng quát của hàm điểm có dạng:

$$S(p) = S_{phone}(p) + S_{receiver}(p) - S_{sender}(p) + S_{layout}(p) + S_{consistency}(p)$$

Trong đó S_{phone} phản ánh tính hợp lệ và chất lượng ký tự; $S_{receiver}$ và S_{sender} phụ thuộc anchor tương ứng; S_{layout} dùng vị trí dòng, box; $S_{consistency}$ phản ánh sự ủng hộ giữa các hướng xoay. Đây là cách mô tả các thành phần, không phải các xác suất đã được hiệu chuẩn.

Sau khi sắp xếp các ứng viên, top.score và margin=top.score−second.score được sử dụng để hạn chế quyết định khi hai ứng viên ngang nhau. Theo code hiện tại, nếu có từ hai ứng viên trở lên, trường hợp top.score < 1 hoặc margin < 1,25 dẫn đến LOW/UNCERTAIN. Trường hợp top.score ≥ 4 và margin ≥ 2 có thể được đánh dấu HIGH. Nếu chỉ có một số, code phân loại HIGH từ score ≥ 4; điều này khác việc chứng minh chắc chắn số đó thuộc người nhận và cần được kiểm tra bằng ngữ cảnh để tránh chấp nhận một bill chỉ chụp số người gửi. [5]

3.3.5. Ví dụ chọn số điện thoại người nhận

Giả sử OCR trả các dòng: “NGƯỜI GỬI”, “Nguyễn Văn A”, “SĐT: 0909123456”, “NGƯỜI NHẬN”, “Trần Thị B”, “SĐT: 0327622447”. Cả 0909123456 và 0327622447 đều là chuỗi hợp lệ về hình thức. Nếu chọn số xuất hiện trước, hệ thống sẽ chọn nhầm người gửi. V4 nhận ra khối “NGƯỜI NHẬN” nằm gần số 0327622447 hơn, đồng thời số 0909123456 bị ảnh hưởng bởi anchor “NGƯỜI GỬI”. Kết quả kỳ vọng là 0327622447.

Ví dụ này có tính minh họa logic chứ không phải bằng chứng một ảnh cụ thể đã được mô hình nhận đúng. Để chứng minh thực nghiệm, báo cáo cần đi kèm ảnh bill, ground truth, OCR lines, điểm từng candidate và kết quả cuối đối với một case thực sự được chạy trong ứng dụng.

3.3.6. Các mức kết quả và nguyên tắc an toàn nghiệp vụ

Bảng 3.3. Trạng thái kết quả của V4

Trạng thái	Ý nghĩa theo code	Hành động UI hợp lý
HIGH	Điểm chọn mạnh theo ngưỡng V4	Hiển thị kết quả, vẫn cho sửa tay
MEDIUM	Có phone nhưng biên điểm / bối cảnh chưa vượt ngưỡng cao	Hiển thị kèm cảnh báo kiểm tra
LOW / UNCERTAIN	Thiếu phone tin cậy hoặc điểm cạnh tranh sát nhau	Không tự chốt SĐT; đề nghị chụp lại/sửa tay

3.4. Xử lý ảnh nghiêng và xoay

3.4.1. Phân biệt xoay và lệch phối cảnh

Ảnh xoay 90°/180°/270° là biến đổi rời rạc quanh tâm ảnh; chữ có thể nằm ngang sai chiều. Ảnh nghiêng 10–30° là dạng quay liên tục trong mặt phẳng, có thể sửa bằng ước lượng deskew. Ảnh chụp xiên tạo biến dạng phối cảnh, khác bản chất với xoay: chiều cao ký tự, độ rộng dòng và góc song song có thể thay đổi theo vị trí. Một bộ deskew 2D không thể bảo đảm loại bỏ đầy đủ méo phối cảnh.

Bài toán triển khai cần ưu tiên chất lượng theo thứ tự: tiền xử lý không phá ảnh; đọc hướng chính trước; khi bộ quyết định chưa chắc chắn mới kích hoạt hướng fallback. Những cải tiến có thể tăng tốc nhưng bỏ nhiều box hoặc thay thứ tự inference không phù hợp với ảnh nghiêng phải được đánh giá trên cùng tập góc trước khi giữ lại.

3.4.2. Auto-deskew trong Kotlin

Auto-deskew trong SmartLockerOcrNativeModule.kt sử dụng vùng tìm góc ±35°, bước quét thô 5° và quét tinh quanh góc tốt nhất trong bán kính 4°. Ảnh mẫu đánh giá được giảm kích thước tối đa 448 pixel mỗi chiều và lấy đặc trưng dựa trên vùng tối. Chỉ khi góc lớn hơn hoặc bằng 3° và độ cải thiện điểm từ 1,65 trở lên thì chương trình chấp nhận sửa góc.

Các ngưỡng này phản ánh lựa chọn kỹ thuật trong bản stable, không nên diễn giải thành một bảo đảm chính xác ở mọi ảnh. Nếu ảnh nhiều đường viền, nền tương phản hoặc chữ bị chói, thước đo góc có thể nhiễu. Điều kiện chấp nhận góc giúp giảm khả năng xoay nhầm ảnh vốn đã thẳng, nhưng không thay thế kiểm thử thật.

3.4.3. R0 fast-path và tổng hợp bốn hướng

R0 tương ứng với ảnh hướng 0° sau bước hiệu chỉnh tiền xử lý. Với một bill đặt đúng chiều, V4 có thể trả HIGH ngay sau OCR hướng R0; code Native Speed V3 chấp nhận trả sớm trong trường hợp score đạt điều kiện. Cách này tránh chi phí của những lượt nhận dạng 90°/270°/180° vốn không cần thiết ở bill dễ.

Khi R0 không rõ ràng, hệ thống có thể tiếp tục các hướng xoay và tổng hợp phone candidate giữa nhiều kết quả. Nếu cùng một phone được đọc ổn qua nhiều hướng, điều đó tạo thêm dấu hiệu nhất quán. Nếu hai phone cạnh tranh sát điểm, thuật toán có thể trả UNCERTAIN để người dùng xác minh. Đây là cách cân bằng giữa tốc độ và rủi ro chọn nhầm phone.

3.4.4. Hạn chế của xử lý ảnh nghiêng

Một số mẫu bill chỉ hiển thị người nhận, thiếu hẳn anchor người gửi; ngược lại có bill cắt đúng phần người gửi và tạo ra một phone hợp lệ nhưng sai vai trò. Với ảnh nghiêng mạnh, góc 30° có thể được deskew, nhưng các biến dạng phối cảnh và blur vẫn ảnh hưởng đến Detector. Bởi vậy báo cáo nên công bố hiệu năng theo nhóm chính diện, nghiêng, xoay ngang và phối cảnh, không sử dụng một tỷ lệ trung bình duy nhất để khẳng định tất cả góc đều tốt.

CHƯƠNG 4. KIỂM THỬ, TỐI ƯU VÀ TRIỂN KHAI

4.1. Phương pháp kiểm thử mô hình

4.1.1. Các cấp độ kiểm thử cần phân biệt

Bộ kiểm thử nên có ít nhất bốn cấp: (1) Recognizer trên crop text-line theo nhãn; (2) OCR đầy đủ trên ảnh bill; (3) chọn đúng phone cuối theo ground truth; (4) thời gian và ổn định trên điện thoại thật. Một model có thể đọc ký tự tốt nhưng chọn sai người gửi, hoặc nhận đúng trên Kaggle nhưng chạy chậm trên Android. Vì vậy không được dùng một metric để thay cho các mục tiêu khác.

Theo mô tả của tài liệu, notebook đánh giá có 8 cell nhưng chưa lưu kết quả đầu ra của từng cell. Bộ robustness có 36 nhãn được lập trong ground_truth_FIXED.csv; ZIP ảnh gồm 36 mẫu đánh giá và một ảnh preview, ngoài README/CSV. Bộ unseen có 6 ảnh PNG. Đây là bằng chứng về cấu trúc tập kiểm thử, chưa phải bằng chứng số lượng dự đoán đúng cuối cùng. [3]

4.1.2. Thiết kế robustness 36 trường hợp

CSV ground truth có 36 dòng ứng với bốn bill gốc, mỗi bill có chín kiểu biến thể: original, landscape_90cw, landscape_90ccw, tilt_plus15, perspective_left, perspective_right, receiver_only, top_sender_receiver và receiver_corner. Các nhãn gốc được lưu trong CSV qua trường expected_receiver_phone; khi tính độ chính xác phải so sánh phone dự đoán với cột này. [6]

Bộ test này kiểm tra một phạm vi biến đổi đa dạng nhưng vẫn nhỏ về số bill độc lập. Vì 36 ảnh xuất phát từ bốn bill gốc, tỷ lệ đúng tính theo ảnh không tương đương kết quả trên 36 nhãn vận chuyển độc lập. Do đó nên báo cáo song song số ảnh và số bill nguồn để tránh phóng đại khả năng tổng quát.

Bảng 4.1. Bộ kiểm thử độ bền vững

Dạng biến thể	Số trường hợp từ CSV	Mục đích
original	4	Ảnh gốc
landscape_90cw	4	Xoay 90° thuận
landscape_90ccw	4	Xoay 90° nghịch
tilt_plus15	4	Nghiêng +15°

Dạng biến thể	Số trường hợp từ CSV	Mục đích
perspective_left	4	Phối cảnh từ trái
perspective_right	4	Phối cảnh từ phải
receiver_only	4	Cắt vùng người nhận
top_sender_receiver	4	Hai block trên/dưới
receiver_corner	4	Người nhận ở góc

4.1.3. Unseen test 6 ảnh

Bộ unseen gồm sáu PNG mang tên những kiểu nhãn khác nhau, ví dụ NovaShip, LockerPost, Viet Parcel Hub, SmartLocker, FastShip và ECO Delivery. Mục đích là kiểm tra khả năng xử lý khi bill có layout/font không trùng bộ robustness hoặc dữ liệu fine-tune. Tuy nhiên, tên "unseen" ở đây mô tả bộ kiểm thử được cung cấp; để khẳng định unseen đúng nghĩa học máy còn cần đảm bảo hình nguồn không trùng train và không được dùng để chỉnh thuật toán trong quá trình phát triển.

Notebook đánh giá có chức năng nhận dạng bộ ảnh unseen và tạo các dự đoán phục vụ đối chiếu. Các kết quả cần được xác minh bằng nhãn tham chiếu độc lập, đồng thời phân biệt ảnh chưa có số điện thoại người nhận rõ ràng với trường hợp mô hình nhận dạng sai.

4.1.4. Định nghĩa chỉ số đánh giá

$$\text{Exact} - \text{match accuracy} = \frac{N_{\text{đúng}}}{N_{\text{tổng}}}$$

$$\text{Coverage} = \frac{N_{\text{trả SĐT}}}{N_{\text{tổng}}}$$

$$\text{Selective accuracy} = \frac{N_{\text{đúng}}}{N_{\text{trả SĐT}}}$$

Trong đó $N_{\text{tổng}}$ là tổng số ảnh; $N_{\text{đúng}}$ là số ảnh hệ thống chấp nhận và trả SĐT trùng hoàn toàn expected_receiver_phone; $N_{\text{trả SĐT}}$ là số ảnh hệ thống chấp nhận và trả một SĐT. Ảnh trả UNCERTAIN hoặc trống không được tính vào $N_{\text{trả SĐT}}$. Selective accuracy chỉ xác định khi $N_{\text{trả SĐT}} > 0$. Latency trên Android phải ghi rõ đo từ lúc kích hoạt chụp đến khi form cập nhật hay chỉ đo giai đoạn suy luận, và dùng cùng phạm vi đo khi so sánh.

Với bộ robustness gồm 36 trường hợp, độ chính xác đầu cuối chỉ có thể được kết luận khi so sánh toàn bộ số điện thoại dự đoán với nhãn chuẩn tương ứng. Thông tin có trong notebook cho phép tổ chức phép kiểm thử, nhưng không thay thế bảng kết quả dự đoán đã được ghi nhận từ một lần chạy cụ thể.

4.1.5. Thời gian xử lý trên Android

Trong quá trình triển khai thử nghiệm, thời gian xử lý từng được ghi nhận ở mức một đến hai phút trước khi tối ưu. Với phương án Kotlin Native, đầu ra Recognizer TopK-F32 và cơ chế fast-path, một số ca quan sát đạt khoảng 4–6 giây. Đây là thời gian quan sát trong điều kiện thử nghiệm, không phải kết quả benchmark chuẩn hóa trên toàn bộ bộ dữ liệu; tốc độ phụ thuộc ảnh đầu vào và thiết bị chạy.

Đánh giá hiệu năng có thể dựa trên các thành phần thời gian: xử lý ảnh, Detector, Recognizer, các lượt xoay và tổng thời gian từ thao tác chụp đến khi xuất hiện kết quả. Khi tiến hành benchmark, cần cố định thiết bị, phiên bản build, bộ ảnh và điều kiện thực hiện để kết quả có thể so sánh.

4.2. Tối ưu hiệu năng trên thiết bị

4.2.1. Tối ưu hợp lệ đã đưa vào stable

Phiên bản hiện tại giữ bốn nhóm tối ưu đã được nhóm lựa chọn: tiền xử lý/inference trong Native Kotlin nhằm giảm overhead bridge, Recognizer đầu ra TopK-F32 để giảm chi phí decode, auto-deskew phục vụ ảnh nghiêng và R0 fast-path để hạn chế các lượt xoay không cần thiết. Các thay đổi này đi theo nguyên tắc không sửa trực tiếp trọng số của model gốc khi không có đánh giá chất lượng phù hợp.

Số batch Recognizer chịu ảnh hưởng của lượng text box phát hiện được. Tuy nhiên, giảm số box chỉ có lợi nếu vẫn bao phủ thông tin người nhận. Nỗ lực ưu tiên box Progressive ROI từng gây lỗi nhiều hơn ở ảnh nghiêng, do đó không được dùng trong bản stable. Cách đánh giá đúng là giữ cả tốc độ và độ đúng của trường hợp xấu; cải thiện average latency nhưng tạo thêm sai phone có thể không chấp nhận được đối với nghiệp vụ giao nhận.

4.2.2. Các hướng đã thử nhưng không đưa vào bản chính

Bảng 4.2. Các phương án tối ưu đã khảo sát

Hướng tối ưu	Quan sát khi thử nghiệm	Quyết định
Dynamic / Static INT8	Không đạt quality gate hoặc lợi ích tốc độ không đủ; Static INT8 giảm match nghiêm trọng	Không triển khai
Mobile vocabulary prune	Cắt lớp đầu ra từ ~18,7 nghìn xuống vài trăm, Kaggle benchmark chỉ tăng nhẹ	Không triển khai
Adaptive width 256/224/192	Chạy nhanh hơn baseline nhưng agreement ở bài test tổng hợp thấp	Không triển khai
Progressive ROI	Có tiềm năng giảm số crop, nhưng ảnh nghiêng lỗi nhiều hơn	Quay về Native Speed V3
Fast Recognizer mới	Ý tưởng cần train thêm và đánh giá kỹ, chưa có kết quả	Chưa triển khai

4.2.3. Cân bằng tốc độ và độ chính xác

Trong hệ thống, mục tiêu tối hậu không phải một con số latency nhỏ bất kể hậu quả. Nếu quá trình tiết kiệm hai giây khiến xác suất trả nhầm SĐT người gửi tăng, hệ thống có thể gửi sai thông báo và gây sai nghiệp vụ. Tối ưu hợp lý phải được ràng buộc bởi accuracy/coverage/robustness tối thiểu, đặc biệt các ảnh nghiêng và bill có hai số hợp lệ.

Các hướng tối ưu mới chỉ nên đưa vào bản ứng dụng khi được đánh giá trên cùng tập ảnh và chứng minh không làm tăng nguy cơ nhận nhầm số người gửi. Nhánh thử nghiệm phải có khả năng rollback và không làm thay đổi model đang demo chính thức khi chưa qua bộ test chuẩn.

4.3. Triển khai và bảo mật

4.3.1. Các bước tái lập phần AI

Để tái lập training: tạo Kaggle Notebook GPU, Add Input bản gốc MC-OCR với mcocr_train_df.csv và train_images, chạy 14 cell theo thứ tự, xác minh đường dẫn tới PaddleOCR v3.7.0 và pretrained weights, kiểm tra crop/label trước train, lấy best checkpoint, chạy eval MC-OCR/test phone rồi export ONNX. Nếu runtime môi trường

Kaggle thay đổi trong tương lai, mã có thể cần cập nhật phiên bản phụ thuộc; việc này phải được ghi rõ chứ không thay đổi phương pháp train cốt lõi.

Để kiểm thử offline: Add Input model đã export và ZIP robustness, chạy notebook test 8 cell, kiểm tra đúng 36 nhãn của CSV, lưu kết quả prediction/summaries dạng CSV. Để tích hợp app: đặt detector/recognizer/dictionary đúng vào android assets, đảm bảo Kotlin module được đăng ký, build lại app, chụp bill chính diện/ảnh nghiêng, so sánh phone và thời gian. Không dùng một notebook test để thay thế toàn bộ quá trình đo trên điện thoại.

4.3.2. Quản lý phiên bản và xác minh nguồn

Sau nhiều lần thay đổi cấu hình, chỉ tên file model không đủ để xác định phiên bản suy luận. Mã băm SHA256 của từng thành phần và mô tả cấu hình cho phép đối chiếu model với mã nguồn, đồng thời hỗ trợ phát hiện sai lệch khi cập nhật hệ thống.

Hai thành phần được sử dụng trong kiến trúc đã chọn là "Recognizer V6 TopK-F32" và "Native Speed V3 stable". Những thử nghiệm INT8, Progressive ROI và điều chỉnh input width không thuộc cấu hình triển khai đã chọn.

4.3.3. Quyền riêng tư và bảo mật dữ liệu giao hàng

Số điện thoại, tên, địa chỉ trên bill là dữ liệu cá nhân cần được bảo vệ. Khi lưu ảnh debug, tạo video demo hoặc công bố ảnh trong báo cáo, nhóm nên sử dụng nhãn giả lập hoặc che các trường thật. Các khóa truy cập và thông tin xác thực của backend không nên được công bố cùng phần mã nguồn AI. Trước khi đóng gói để nộp hoặc chia sẻ, cần kiểm tra và loại bỏ khóa truy cập, thông tin xác thực cùng các tệp cấu hình riêng; chỉ đưa vào gói những thành phần cần cho phần AI.

Suy luận trên thiết bị giúp tránh gửi ảnh đến một dịch vụ OCR bên ngoài nhưng không tự động bảo đảm mọi dữ liệu đều không truyền lên mạng: sau khi xác nhận, nghiệp vụ SmartLocker có thể gửi số điện thoại tới backend để tạo đơn và thông báo. Báo cáo cần mô tả đúng luồng lưu/chuyển SĐT và giới hạn truy cập, thay vì khẳng định dữ liệu không bao giờ rời thiết bị.

4.3.4. Hạn chế của phương pháp

Các hạn chế gồm bộ robustness 36 ảnh có chỉ bốn bill nguồn, độ chính xác ảnh phối cảnh chưa được định lượng theo nhóm thống nhất, và hiệu năng phụ thuộc điện

thoại. Việc đo kiểm có tính đại diện rộng hơn là cần thiết để lượng hóa khả năng tổng quát hóa.

Một hạn chế phương pháp cần lưu ý nữa là tầng chọn SĐT V4 dựa trên luật và mức điểm thủ công: dễ kiểm tra, dễ giải thích, nhưng có thể nhạy với nhãn ít gặp, font đặc biệt hoặc bố cục cực lạ. Đây là cơ hội để cải tiến bằng thêm dữ liệu đa dạng, quy tắc đánh giá, hoặc một bộ phân loại ngữ cảnh nhỏ nếu có đủ tập gán nhãn.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Hệ thống được xây dựng theo một pipeline AI có các thành phần rõ ràng: Detector PP-OCRv6 Small pretrained, Recognizer PP-OCRv6 Small được fine-tune trên MC-OCR và tăng cường số điện thoại, export ONNX, inference trên Android Native, cùng thuật toán V4 nhận diện vai trò người nhận theo phone + semantic anchors + scoring. Đầu ra của AI được gắn vào giao diện React Native phục vụ tác vụ giao hàng.

Giá trị nổi bật nằm ở việc kết hợp mô hình học máy với tri thức miền ứng dụng, đồng thời đưa inference đến thiết bị Android. Bài toán khó không chỉ là nhận diện chữ mà là quyết định đúng số điện thoại giữa các chuỗi gần giống nhau. Phiên bản stable được giữ lại sau nhiều thử nghiệm thể hiện nguyên tắc kỹ thuật quan trọng: ưu tiên độ đúng, tính ổn định và khả năng giải thích trước khi chạy theo một KPI tốc độ quá chặt.

5.2. Hướng phát triển

Về dữ liệu, nên tăng số bill độc lập từ nhiều nhà vận chuyển, bổ sung ảnh mờ/chói và phối cảnh nhiều mức, đồng thời ghi nhãn cặp sender/receiver đầy đủ. Về thuật toán, có thể nghiên cứu rectification phối cảnh tốt hơn, confidence calibration dựa trên tập validation và phát hiện nhãn tương ứng nhiều ngôn ngữ. Về hiệu năng, chỉ áp dụng một tối ưu mới sau khi so sánh paired benchmark trên cùng ảnh/điện thoại và kiểm tra trường hợp nghiêng không giảm độ đúng.

Hướng ứng dụng tương lai là chuẩn hóa giao thức kết quả AI (phone, confidence, provenance, processing time, warning) để backend/ứng dụng có thể ghi nhận quyết định và hỗ trợ audit khi người dùng báo sai. Những cải tiến này biến công trình từ một chương trình nhận dạng OCR thành một mô-đun nhận dạng có thể bảo trì, kiểm thử và nâng cấp lâu dài.

TÀI LIỆU THAM KHẢO

[1] PaddlePaddle, PP-OCRv6 Documentation. https://github.com/PaddlePaddle/PaddleOCR/blob/main/docs/version3.x/algorithm/PP-OCRv6/PP-OCRv6.en.md.

[2] SmartLocker_PP-OCRv6_MC-OCR_FULL_14_CELLS_FIXED_V2.ipynb. Tài liệu nội bộ. Notebook chuẩn bị dữ liệu, fine-tune Recognizer, đánh giá và xuất mô hình ONNX.

[3] SMARTLOCKER_TEST_FULL_8_CELLS_V4_HYBRID.ipynb. Tài liệu nội bộ. Notebook đánh giá OCR và thuật toán lựa chọn số điện thoại V4.

[4] SmartLockerOcrNativeModule.kt. Tài liệu nội bộ. Mã nguồn tiền xử lý ảnh và suy luận PP-OCRv6 trên Android.

[5] smartlockerOcr.ts. Tài liệu nội bộ. Mã nguồn trích xuất và chấm điểm số điện thoại người nhận.

[6] ground_truth_FIXED.csv; SMARTLOCKER_ROBUSTNESS_TEST_SET_FIXED.zip; SMARTLOCKER_UNSEEN_TEST_SET.zip. Tài liệu nội bộ. Bộ dữ liệu phục vụ kiểm thử.

[7] RIVF2021 MC-OCR Organizers, Mobile-Captured Image Document Recognition for Vietnamese Receipts, 2021. https://www.rivf2021-mc-ocr.vietnlp.com/.

[8] A. Graves, S. Fernández, F. Gomez và J. Schmidhuber, Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks, ICML, 2006, tr. 369–376. https://www.cs.toronto.edu/~graves/icml_2006.pdf.

[9] Microsoft, ONNX Runtime Java and Android Documentation. https://onnxruntime.ai/docs/get-started/with-java.html.

[10] React Native, Android Native Modules. https://reactnative.dev/docs/legacy/native-modules-android.

PHỤ LỤC A. NỘI DUNG NOTEBOOK HUẤN LUYỆN

Bảng A.1 tóm tắt 14 công đoạn xử lý được tổ chức trong notebook huấn luyện PP-OCRv6 Small. Thứ tự này phản ánh luồng chuẩn bị dữ liệu, fine-tune và chuyển mô hình phục vụ suy luận.

Bảng A.1. Các công đoạn notebook huấn luyện

Cell	Tên công đoạn	Đầu ra / mục đích
01	Settings + GPU check	Seed, cờ augmentation, epoch, thống kê GPU
02	Cài PaddleOCR v3.7.0	Chuẩn bị mã huấn luyện, phụ thuộc
03	Locate MC-OCR	Tìm CSV và ảnh gốc, kiểm tra cột bắt buộc
04	Download pretrained	Detector/Recognizer weights ban đầu
05	Build text-line dataset	Warp polygon, crop, nhãn, train/val/test
06	Visual check	Kiểm tra tương thích crop và text
07	Export Detector legacy IR	Giữ pretrained, không fine-tune
08	Recognizer Stage A	10 epoch trên MC-OCR
09	Best checkpoint Stage A + eval	Lựa model và đánh giá
10	Phone augmentation + Stage B	4 epoch, 4.000/800 mẫu bổ sung
11	Final evaluation	MC-OCR real test và synthetic phone holdout
12	Export Recognizer legacy IR	Tạo Paddle inference
13	Paddle2ONNX	Chuyển det/rec sang ONNX
14	Package final OCR engine	Assets, checkpoint, YAML, manifest

PHỤ LỤC B. GIẢ MÃ LUỒNG NHẬN DIỆN

Giả mã dưới đây biểu diễn luồng chính từ ảnh bill đến số điện thoại người nhận. Các ngưỡng quyết định và xử lý ngoại lệ được xác định cụ thể trong mã nguồn triển khai.

```
function READ_RECEIVER_PHONE(image):
    bitmap = decode_resize_and_deskew(image)
    results = []
    for angle in [0, 90, 270, 180]:
        oriented = rotate(bitmap, angle)
        boxes = detector_PP_OCRv6(oriented)
        lines = recognizer_TopK_F32(boxes)
        candidates = extract_vietnam_phone(lines)
        result = receiver_selector_V4(candidates, lines)
        results.append(result)
        if angle == 0 and result.confidence == HIGH:
            return result.phone
    best = aggregate_results(results)
    return best.phone if best.accepted else UNCERTAIN
```